Homework 7: Async Order Processing (Part II and Part III)

Field	Santosh Vasisht Sunkara	Sameep Anil Sawant
NUID	002593529	002535931
Repository	https://github.com/santosh50/CS6650_2b_demo/tree/master/HW07	https://github.com/samsaw777/BSDS_Course_Work/tree/main/HW07
Course	CS 6650 Building Scalable Distributed Systems, Northeastern University	same
Report date	November 4, 2026	November 4, 2026

1. Overview and setup

- **Problem:** each order needs a 3 s payment verification and the payment processor handles one order at a time. Under flash sale load, a synchronous API makes customers wait in line, and throughput caps at about 0.33 orders/s.
- **Implemented (both):** a synchronous order API (payment inside the request), an asynchronous order API (Receiver → SNS → SQS → Processor workers), and an SNS-triggered Lambda processor for comparison.
- **Deployment (both):** Go services on AWS ECS Fargate in us-east-1 (Learner Lab, LabRole), provisioned with Terraform. VPC 10.0.0.0/16, public subnets 10.0.1.0/24 and 10.0.2.0/24 for the ALB, private subnets 10.0.10.0/24 and 10.0.11.0/24 for ECS. Tasks use 256 CPU units and 512 MB, with a /health endpoint.

Setting	Santosh	Sameep
Payment gate	Buffered channel, capacity 1	Buffered channel, capacity 1
Locust profile	wait 0.1–0.5 s; no client timeout; run from a laptop in Boston	wait 0.1–0.5 s; no client timeout; run from Boston
Worker count control	terraform apply -var worker_count=N	terraform apply -var processor_workers=N
Processor log (every 10 s)	workers, processed, in_flight, rate_10s, avg_rate	workers, processed, in_flight, avg_rate
Queue depth source	AWS CLI ApproximateNumberOfMessages every ~11 s, plus CloudWatch (1 min)	CloudWatch ApproximateNumberOfMessagesVisible (1 min)
CloudWatch statistic	Maximum, 1-minute period	Average, 1-minute period
Processor poll design	Only requests as many messages as there are free workers (≤ 10 per call)	Fixed pool of N workers

2. Phase 1: Synchronous system

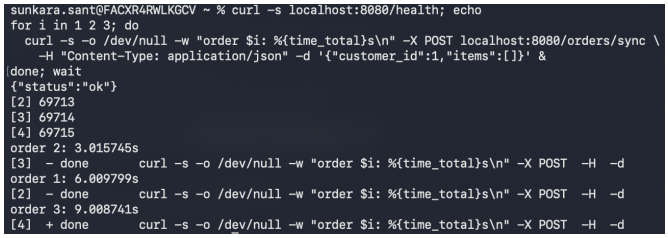
Test setup (both): Locust posting to POST /orders/sync through the ALB. Normal test: 5 users, spawn rate 1, 30 s. Flash sale test: 20 users, spawn rate 10, 60 s.

Gate check (local): 3 concurrent sync requests. Santosh: 3.015745 s, 6.009799 s, 9.008741 s. Sameep: 3.02 s, 6.02 s, 9.02 s. Both gates allow one payment at a time.

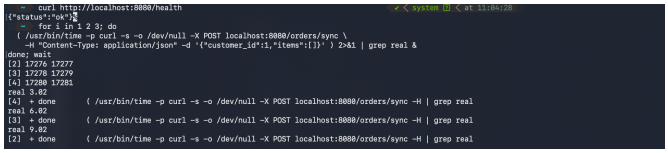
Run	Test	Requests	Failures	Median	p95	Max	Throughput
Santosh	Normal (5 users, 30 s)	9	0	12000 ms	15000 ms	14930 ms	9 ÷ 30 s = 0.3/s

Run	Test	Requests	Failures	Median	p95	Max	Throughput
Sameep	Normal (5 users, 30 s)	9	0	12000 ms	15000 ms	14837 ms	~0.33/s
Santosh	Flash (20 users, 60 s)	19	0	30000 ms	56000 ms	56164 ms	19 ÷ 60 s = 0.3167/s
Sameep	Flash (20 users, 60 s)	19	0	30000 ms	56000 ms	56163 ms	~0.32/s

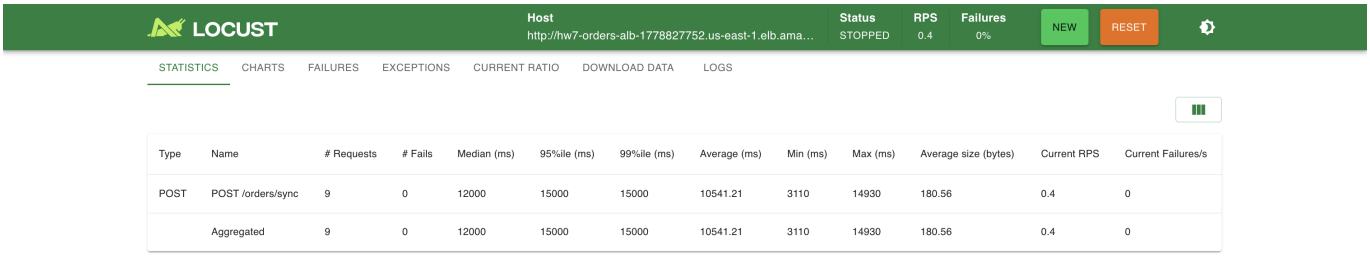
Santosh's extra detail: normal average 10541.21 ms (min 3110 ms, p99 15000 ms); flash average 29665.12 ms (min 3106 ms, p99 56000 ms).



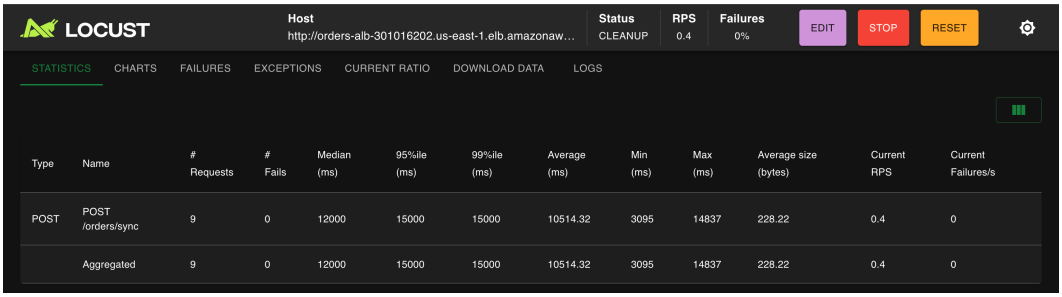
Santosh · Fig 1a. Local gate check: 3.02 / 6.01 / 9.01 s



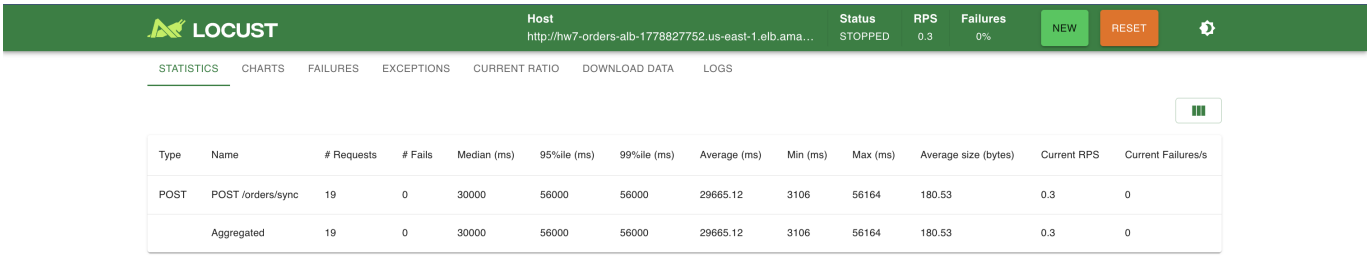
Sameep · Fig 1b. Local gate check: 3.02 / 6.02 / 9.02 s



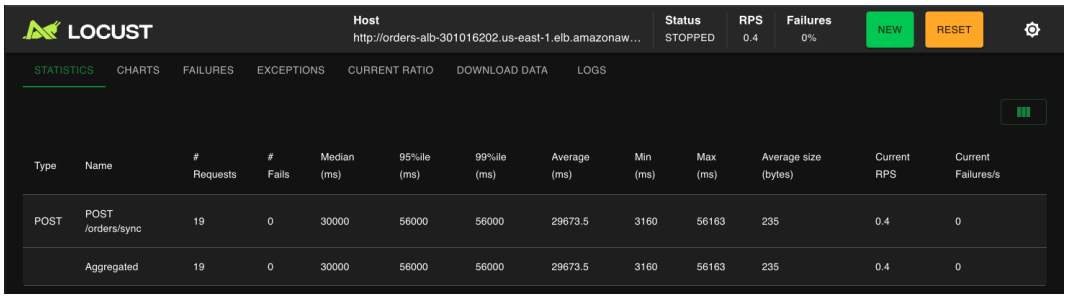
Santosh · Fig 2a. Sync normal test: 9 requests, median 12000 ms, max 14930 ms



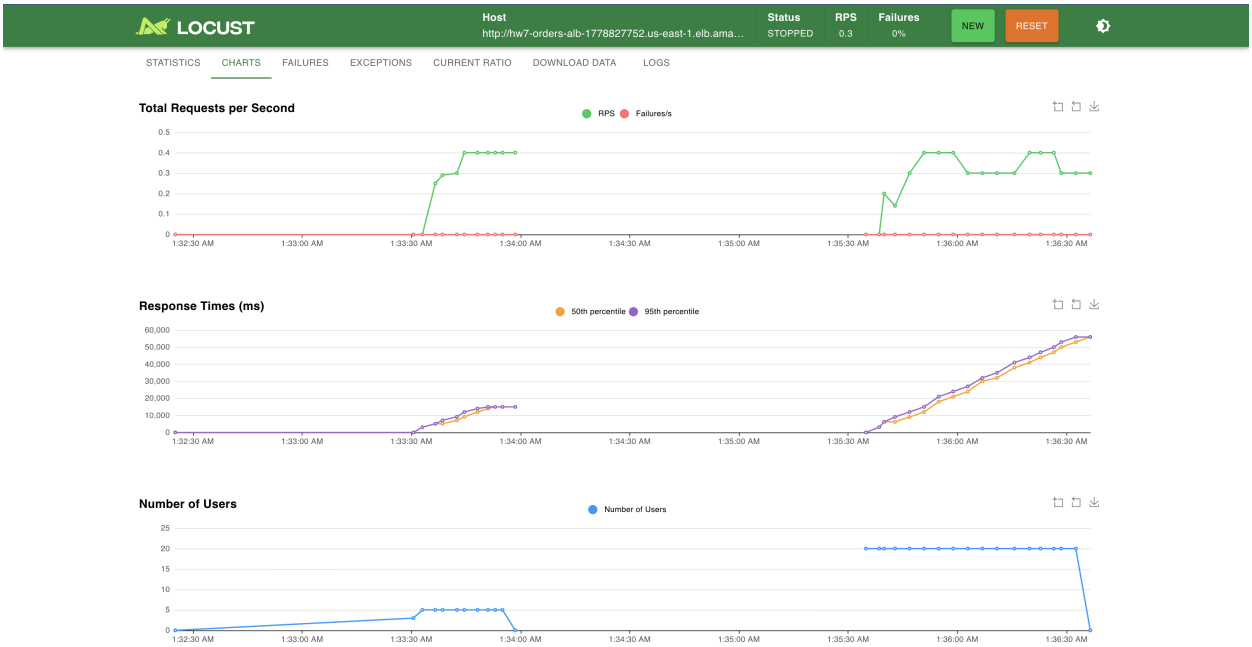
Sameep · Fig 2b. Sync normal test: 9 requests, median 12000 ms, max 14837 ms



Santosh · Fig 3a. Sync flash test: 19 requests, median 30000 ms, max 56164 ms



Sameep · Fig 3b. Sync flash test: 19 requests, median 30000 ms, max 56163 ms



Santosh · Fig 4a. Locust charts for both sync runs: RPS flat at 0.3–0.4 while response time climbs to ~15 s, then ~56 s



Sameep · Fig 4b. Locust charts for both sync runs: RPS ~0.4, response time rising to ~15 s and ~56 s

Observations (both):

- Throughput stayed at ~0.3/s with 4× more users. Extra users only made the line longer.
- Max wait grows with users × 3 s: ~15 s at 5 users, ~56 s at 20 users.

- 0% failures is misleading. With no client timeout, customers waited up to 56 s instead of failing. That is close to the ALB's 60 s idle timeout, so a slightly longer test would return HTTP 504s.
- Requests still waiting when the test stopped were not counted, so 19 understates the customers affected.

Comparison: our sync results are practically identical (same request counts and medians; max within 1 ms on the flash test). The single-slot gate behaves the same in both implementations, which is expected because nothing about the network or AWS setup changes the 3 s serial bottleneck.

3. Phase 2: Bottleneck analysis

Question	Theoretical	Santosh (measured)	Sameep (measured)
Max throughput with 20 customers	1 order / 3 s = 0.33 orders/s	19 / 60 s = 0.3167/s	19 / 60 s = ~0.32/s
Orders lost per second at 20/s demand	20 − 0.33 ≈ 19.67 orders/s	0 failures; customers waited up to 56164 ms	Users wait in line instead of being dropped

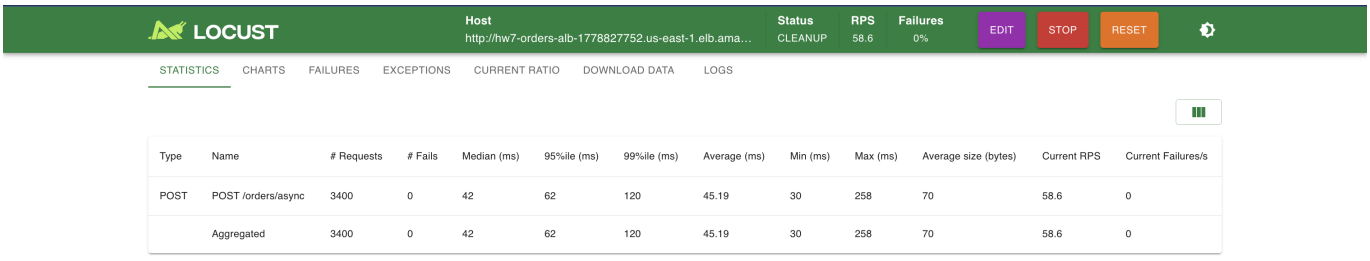
- The measured rate matches theory for both of us. One payment slot caps throughput no matter how many customers arrive.
- **What can change:** payment time cannot be reduced, but the customer does not need to wait for it. Accept the order immediately and verify payment later. That is the async design.

4. Phase 3: Async solution

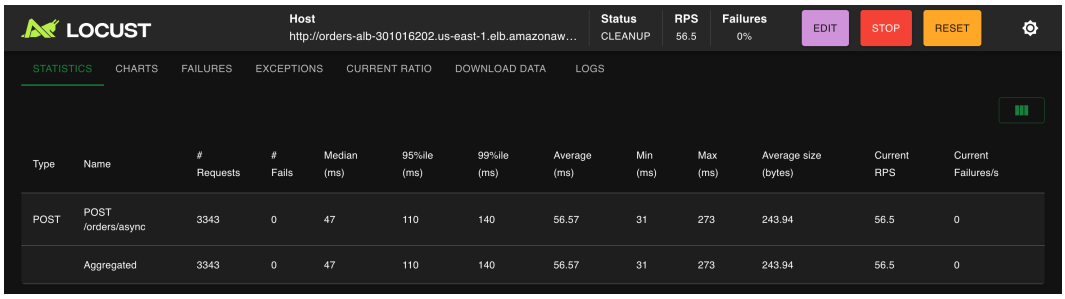
Test setup (both): same flash profile (20 users, spawn rate 10, 60 s), Locust posting to POST /orders/async, processor running 1 worker.

Run	Accepted	Failures	Median	p95	p99	Accept rate	vs sync
Santosh	3400	0	42 ms	62 ms	120 ms	3400 ÷ 60 s = 56.67/s	178.95× (3400 ÷ 19)
Sameep	3343	0	47 ms	110 ms	140 ms	~56/s	~176× (3343 ÷ 19)

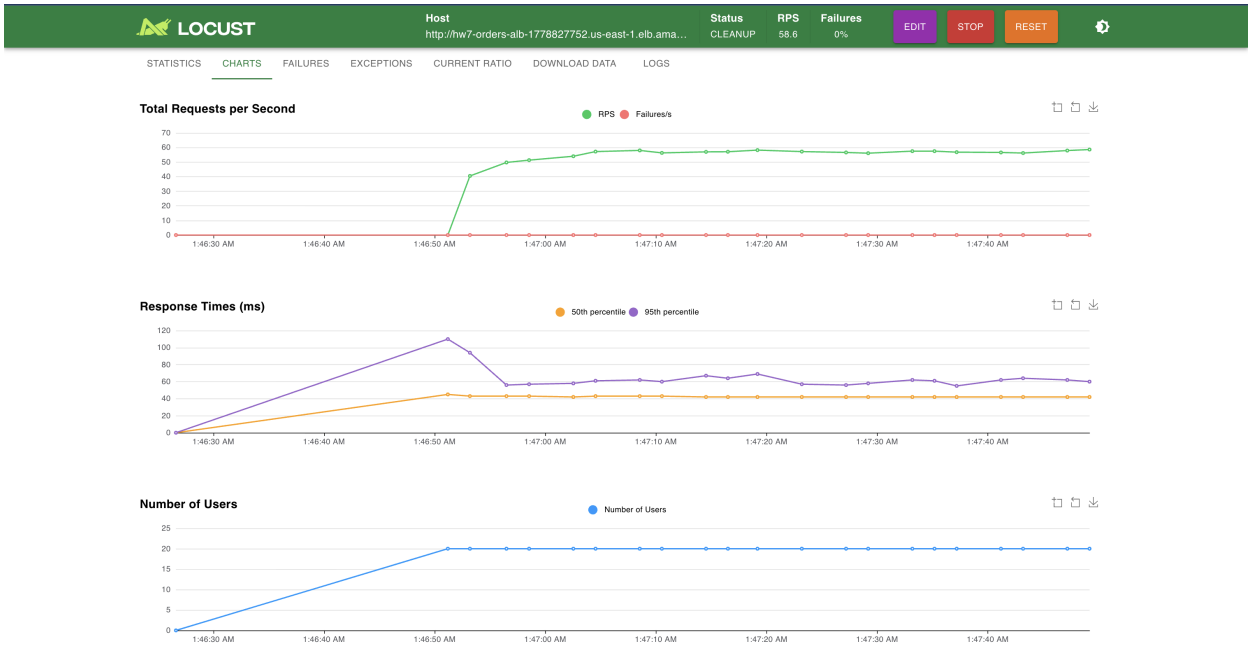
- Both of us got 0 failures and every order accepted in well under a second.
- Santosh: median 42 ms and p95 62 ms, both under the 100 ms target. p95 spiked to ~110 ms only while users ramped up, then settled at 55–70 ms. The Locust UI stayed in CLEANUP after the run; the stats did not change for 3 minutes, so they are final.
- Sameep: median 47 ms meets the target; p95 110 ms is slightly above it, attributed to network latency from Boston plus the SNS publish call. His final Locust HTML report shows 3,346 (a few requests completed during cleanup).



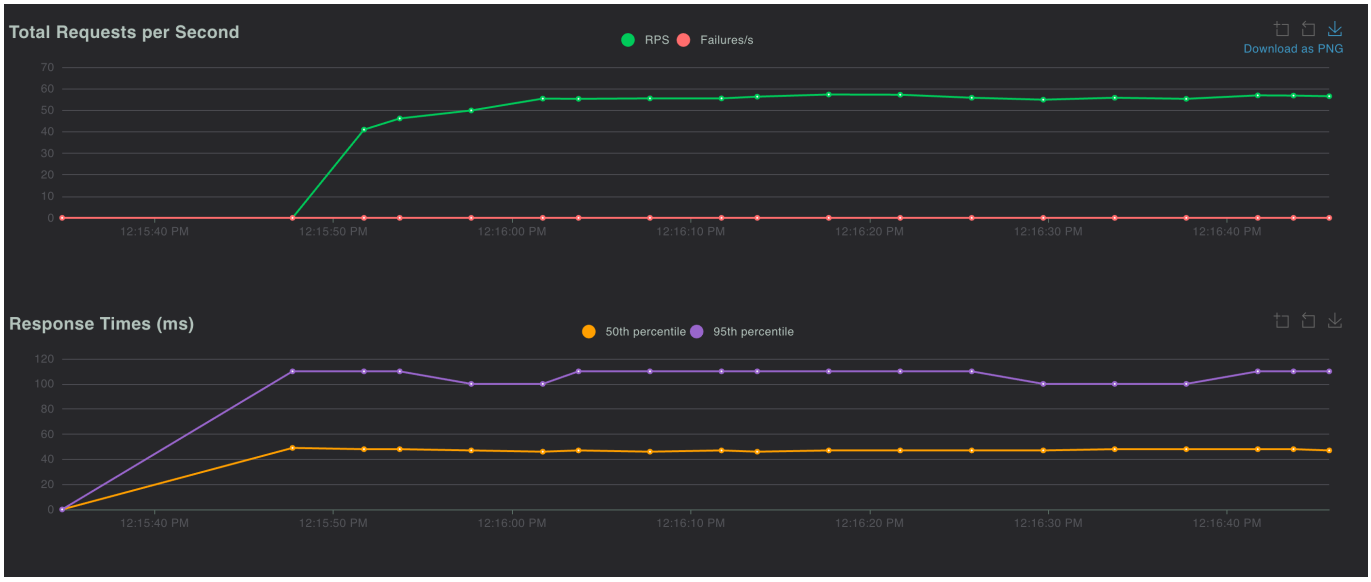
Santosh · Fig 5a. Async flash test, 1 worker: 3400 requests, 0 failures, median 42 ms, p95 62 ms



Sameep · Fig 5b. Async flash test, 1 worker: 3343 requests, 0 failures, median 47 ms, p95 110 ms



Santosh · Fig 6a. Async charts: ~55–58 RPS, median ~42 ms, p95 ~55–70 ms after the ramp-up spike



Sameep · Fig 6b. Async charts: ~56 RPS, median ~47 ms, p95 ~110 ms

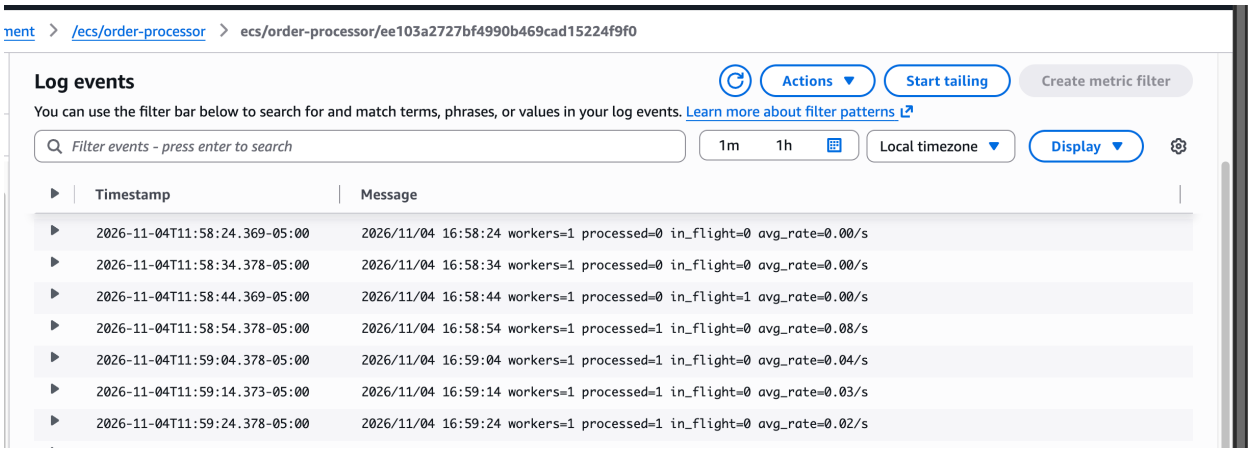
Accepted is not processed

Stage	Description	Santosh (1 worker, 60 s)	Sameep (1 worker, 60 s)
1. Accepted	Receiver publishes to SNS and returns HTTP 202	3400 orders, 0 failures	3,343 orders, 0 failures
2. Processed	Processor pulls from SQS and runs the 3 s payment	20 orders by end of test	~20 orders by end of test

- Locust only measures stage 1: “3400 requests, 0 failures” means 3400 orders accepted, not 3400 paid.
- In the sync design both stages happen in the same request, so the Locust count (19) was also the processed count.
- Measuring the async system needs CloudWatch queue metrics and the processor log.

```
2026-11-09T06:28:00.044000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:00 workers=1 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T06:28:10.044000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:10 workers=1 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T06:28:20.044000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:20 workers=1 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T06:28:30.039000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:30 workers=1 processed=1 in_flight=0 rate_10s=0.10/s avg_rate=0.11/s
2026-11-09T06:28:40.044000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:40 workers=1 processed=1 in_flight=0 rate_10s=0.00/s avg_rate=0.05/s
```

Santosh · Fig 7a. Smoke test: processed goes from 0 to 1 at 06:28:30 UTC



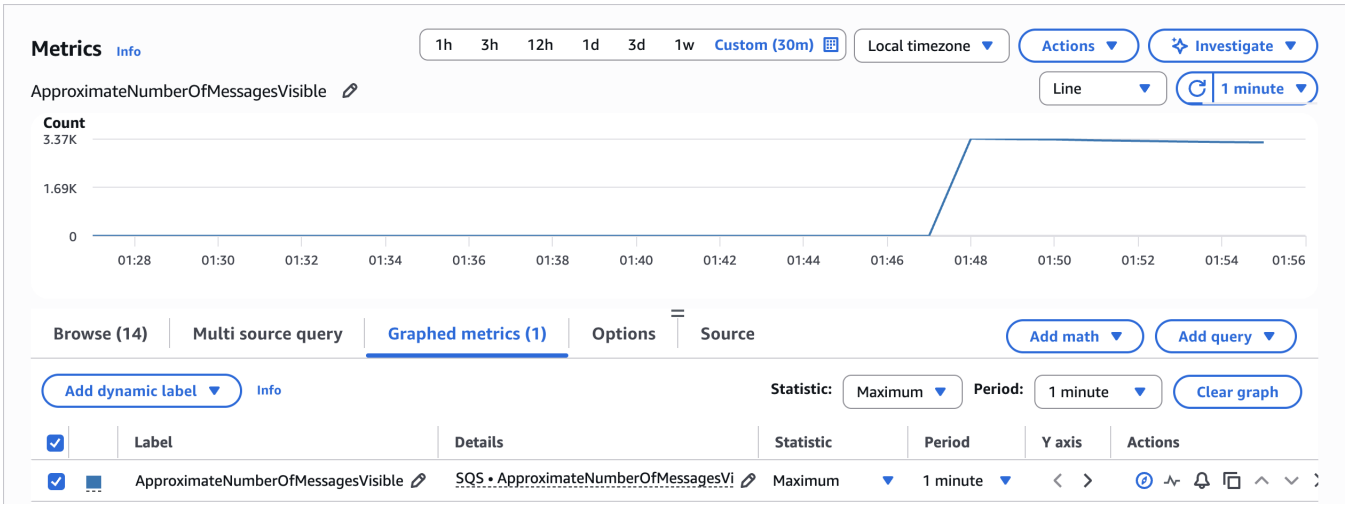
Sameep · Fig 7b. Processor log for the first test order: processed goes from 0 to 1

Comparison: acceptance is the same shape for both of us (~56/s, 0 failures, ~176–179× more than sync). The p95 gap (62 vs 110 ms) comes from the client side: network path and time of day for the laptop running Locust. Server-side work is identical (one SNS publish).

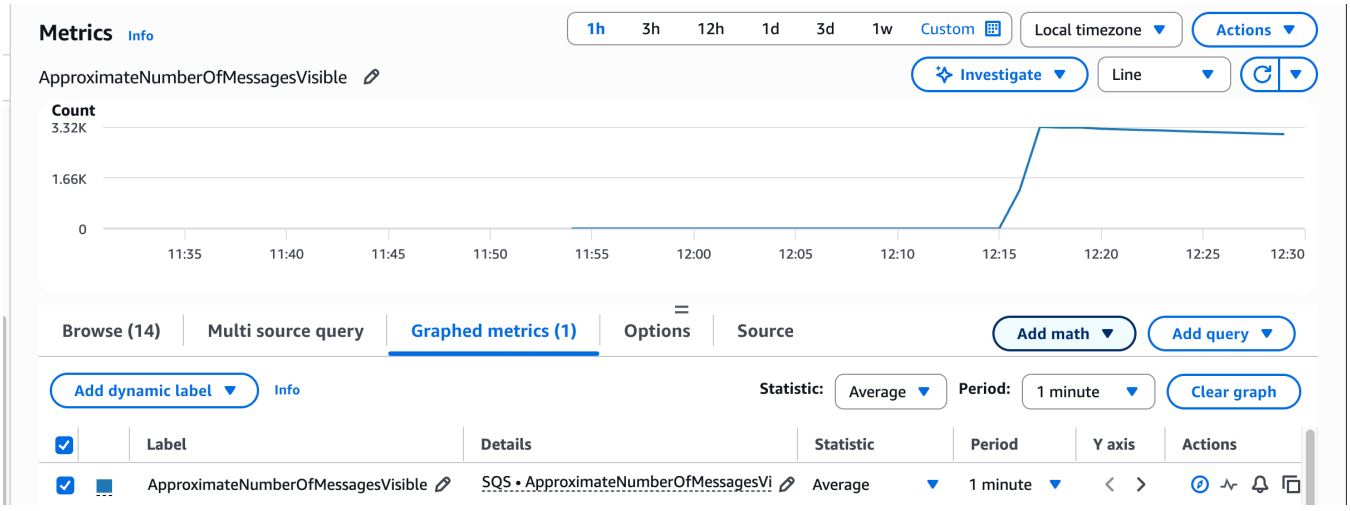
5. Phase 4: Queue buildup (1 worker)

Item	Santosh	Sameep
Peak queue depth	3378 at 01:48:08 local (CLI); = 3400 − 20 processed − 1 in flight	~3,320 right after the test (CloudWatch)
Measured processing rate	0.3303/s (109 orders in 330 s, log 06:47:50–06:53:20 UTC); 0.3348/s over 11 min before purge	~0.35/s (~340 in 16 min; queue 2,984 after 16 min)
Order acceptance rate	56.67/s measured (assignment assumes ~60)	~56/s measured
Queue growth rate	56.67 − 0.3303 = 56.34 messages/s	56 − 0.33 = ~55.7 messages/s
Time to clear actual backlog	3378 ÷ 0.3303 = 10,227 s ≈ 170.5 min (~2.84 h)	3,320 ÷ 0.33 = ~10,000 s ≈ 167 min (~2.8 h)
Full 1-hour flash sale at 60/s (theoretical)	59.67 × 3,600 ≈ 214,800 messages ÷ 0.33 ≈ 10,740 min (~7.5 days)	~214,800 messages ÷ 0.33 ≈ ~10,700 min (~7.5 days)
Time to zero	Not measured: purged at 01:59:08 local	Not measured: purged before Phase 5

- The worker was busy the whole time but far too slow: orders arrived ~170× faster than one worker could pay for them.
- No orders were lost. The queue stores work but does not speed it up.
- The 1-hour case takes longer to clear than SQS's 4-day retention, so with one worker orders would expire unprocessed.
- Santosh's CLI readings jumped around during load (761, 0, 2005, 1847, 2628…). ApproximateNumberOfMessages is approximate: SQS spreads the queue across servers and each call can count a different subset. Readings settle once load stops.
- **Customer impact:** orders are accepted instantly but confirmed hours or days later ("Where's my order confirmation?").



Santosh · Fig 8a. ApproximateNumberOfMessagesVisible (Maximum, 1 min), 1 worker: jumps to ~3.37K at 01:48, then an almost flat decline



Sameep · Fig 8b. ApproximateNumberOfMessagesVisible, 1 worker: spike to ~3.32K after the 12:15 test, then an almost flat decline

Comparison: peaks differ by ~58 messages, mostly because Santosh accepted 57 more orders (3400 vs 3343). Sameep's 16-minute rate (~0.35/s) is slightly above the 1-per-3 s limit; Santosh's two windows (0.3303/s and 0.3348/s) bracket it. Both confirm one worker ≈ 0.33/s.

6. Phase 5: Scaling workers

Test setup (both): async flash test (20 users, spawn rate 10, 60 s) repeated with 1, 5, 20 and 100 worker goroutines in the same 256 CPU / 512 MB task. Santosh ran them in the order 1, 100, 20, 5, waiting for aws ecs wait services-stable and for only the new task to be logging before each run.

Request acceptance (Locust)

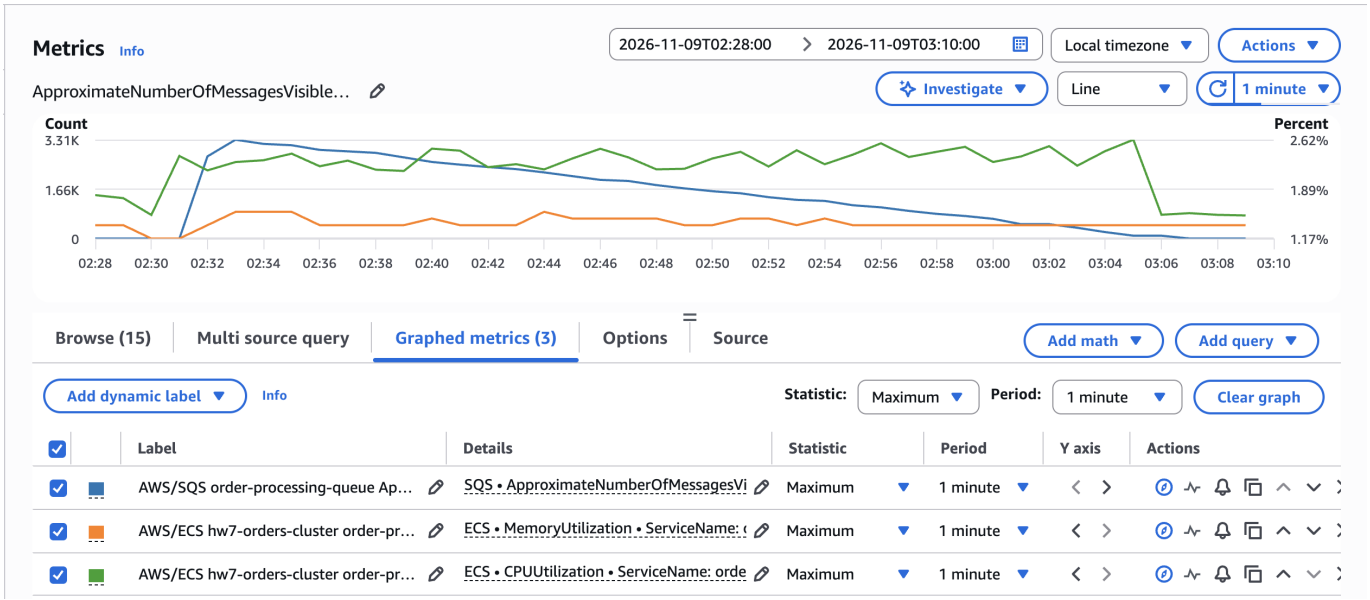
Workers	Santosh: accepted / fails / median / p95	Sameep: accepted / fails / median / p95
1	3400 / 0 / 42 ms / 62 ms	3,343 / 0 / 47 ms / 110 ms
5	3448 / 0 / 43 ms / 60 ms	3,320 / 0 / 49 ms / 110 ms
20	3453 / 0 / 43 ms / 59 ms	3,318 / 0 / 48 ms / 110 ms
100	3431 / 0 / 45 ms / 61 ms	3,361 / 0 / 47 ms / 110 ms

For both of us, acceptance does not depend on worker count. Only the processing side changes.

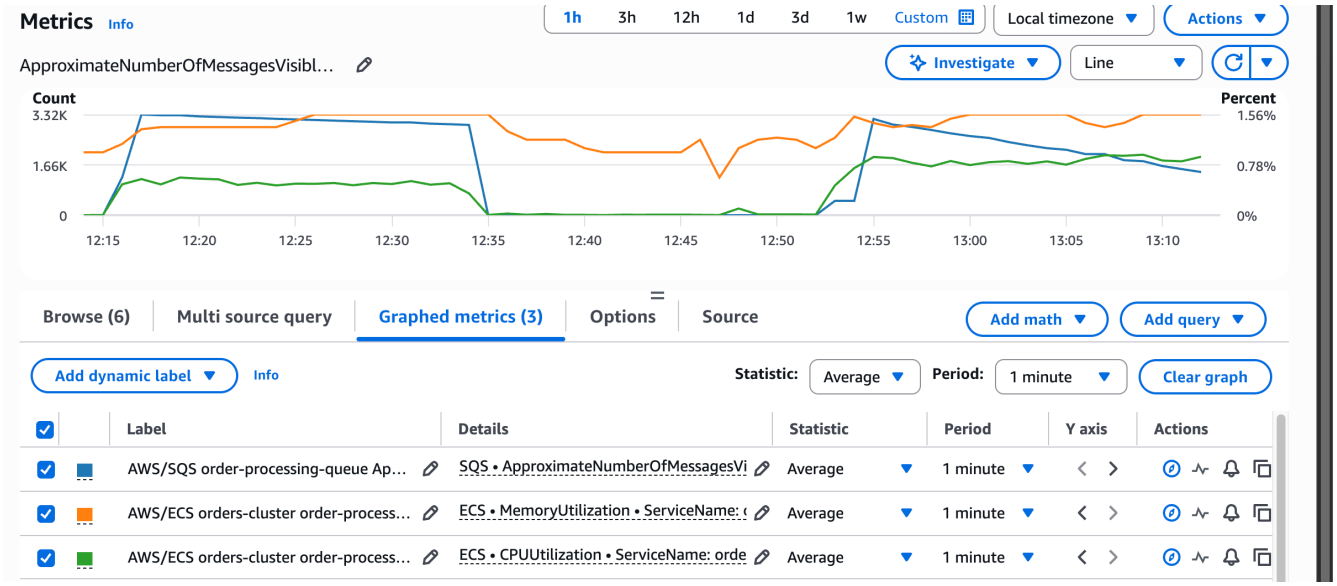
Processing results

Workers	Run	Theoretical rate	Measured rate	Peak depth	Time to zero after load stopped	Accepted / processed
1	Santosh	0.33/s	0.3303/s (99.1%)	3378	Not measured (purged); ~170.5 min projected	3400 / 247 before purge
1	Sameep	0.33/s	~0.35/s	~3,320	Not measured (purged)	3,343 / —
5	Santosh	1.67/s	1.650/s (99.0%)	3345	~33 min 52 s (measured)	3448 / 3450
5	Sameep	1.67/s	~1.65/s	~3,057+	~31 min (projected)	3,320 / —
20	Santosh	6.67/s	6.588/s (98.8%)	2974	~7 min 54 s (measured)	3453 / 3454
20	Sameep	6.67/s	~6.5/s	~2,900	Not measured	3,318 / —
100	Santosh	33.33/s	33.22/s (99.7%)	1352	~1 min (measured)	3431 / 3431
100	Sameep	33.3/s	~31.5/s	~1,250	~1–2 min	3,361 / —

- **Santosh’s measured rates** come from the change in processed over steady windows: 5 workers 3416 in 2070 s, 20 workers 3360 in 510 s, 100 workers 2990 in 90 s. Peak depth comes from the CLI loop.
- **Sameep’s measured rates** come from the processor avg_rate log line and the change in processed per 10 s; peak depth from the CloudWatch graph and accepted − processed at test end.
- Each 4–5× increase in workers cut drain time by about the same factor.
- Santosh: processed was slightly higher than accepted for 5 and 20 workers (+2, +1). That fits SQS’s at-least-once delivery (a rare duplicate), or requests Locust cut off at stop after the receiver had already published them.



Santosh · Fig 9a. 5-worker run alone (02:28–03:10 local): queue jumps to ~3.31K at 02:33 and drains in a near-straight line to 0 by ~03:07; CPU (green) and memory (orange) flat



Sameep · Fig 9b. Queue depth (blue), memory (orange) and CPU (green) for the 1 and 5 worker runs; purge at about 12:34

```
2026-11-09T07:31:22.786000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:31:22 workers=5 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:31:32.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:31:32 workers=5 processed=0 in_flight=5 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:31:42.781000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:31:42 workers=5 processed=15 in_flight=5 rate_10s=1.50/s avg_rate=1.35/s
2026-11-09T07:31:52.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:31:52 workers=5 processed=30 in_flight=5 rate_10s=1.50/s avg_rate=1.42/s
2026-11-09T07:32:02.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:32:02 workers=5 processed=50 in_flight=5 rate_10s=2.00/s avg_rate=1.61/s
2026-11-09T07:32:12.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:32:12 workers=5 processed=65 in_flight=5 rate_10s=1.50/s avg_rate=1.58/s
2026-11-09T07:32:22.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:32:22 workers=5 processed=80 in_flight=5 rate_10s=1.50/s avg_rate=1.57/s
```

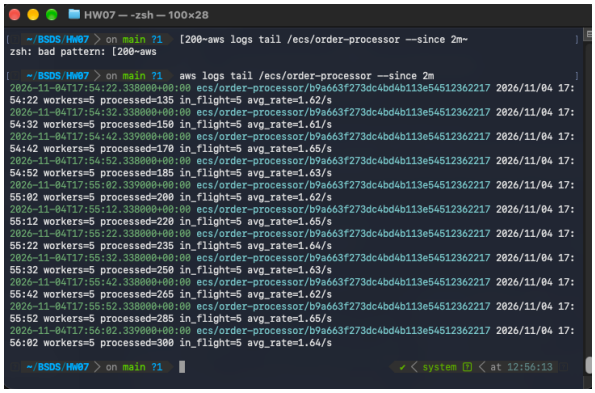
Santosh · Fig 10a. 5 workers: in_flight=5, rate_10s 1.5–2.0/s

```
2026-11-09T07:16:49.159000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:16:49 workers=20 processed=0 in_flight=13 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:16:59.160000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:16:59 workers=20 processed=60 in_flight=20 rate_10s=6.00/s avg_rate=5.79/s
2026-11-09T07:17:09.160000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:09 workers=20 processed=120 in_flight=20 rate_10s=6.00/s avg_rate=5.89/s
2026-11-09T07:17:19.160000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:19 workers=20 processed=190 in_flight=10 rate_10s=7.00/s avg_rate=6.26/s
2026-11-09T07:17:29.159000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:29 workers=20 processed=260 in_flight=20 rate_10s=7.00/s avg_rate=6.44/s
2026-11-09T07:17:39.159000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:39 workers=20 processed=320 in_flight=20 rate_10s=6.00/s avg_rate=6.35/s
2026-11-09T07:17:49.159000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:49 workers=20 processed=380 in_flight=20 rate_10s=6.00/s avg_rate=6.30/s
```

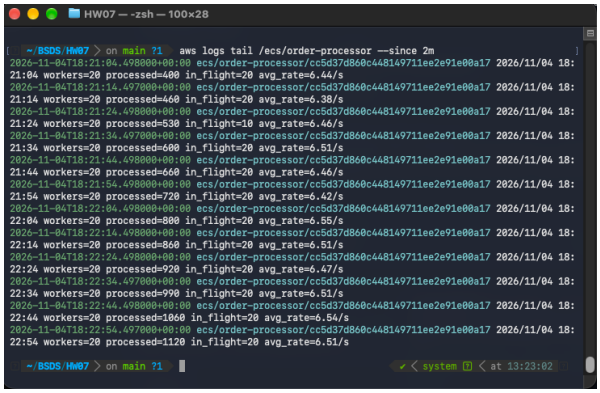
Santosh · Fig 11a. 20 workers: in_flight mostly 20, rate_10s 6–7/s

```
2026-11-09T07:06:18.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:18 workers=100 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:06:28.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:28 workers=100 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:06:38.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:38 workers=100 processed=100 in_flight=100 rate_10s=10.00/s avg_rate=18.99/s
2026-11-09T07:06:48.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:48 workers=100 processed=410 in_flight=100 rate_10s=31.00/s avg_rate=26.86/s
2026-11-09T07:06:58.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:58 workers=100 processed=735 in_flight=95 rate_10s=32.50/s avg_rate=29.09/s
2026-11-09T07:07:08.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:08 workers=100 processed=1092 in_flight=94 rate_10s=35.70/s avg_rate=30.97/s
2026-11-09T07:07:18.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:18 workers=100 processed=1400 in_flight=100 rate_10s=30.80/s avg_rate=30.93/s
2026-11-09T07:07:28.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:28 workers=100 processed=1720 in_flight=100 rate_10s=32.00/s avg_rate=31.12/s
2026-11-09T07:07:38.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:38 workers=100 processed=2066 in_flight=100 rate_10s=34.60/s avg_rate=31.65/s
2026-11-09T07:07:48.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:48 workers=100 processed=2400 in_flight=100 rate_10s=33.40/s avg_rate=31.89/s
2026-11-09T07:07:58.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:58 workers=100 processed=2700 in_flight=100 rate_10s=30.00/s avg_rate=31.67/s
2026-11-09T07:08:08.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:08:08 workers=100 processed=3043 in_flight=97 rate_10s=34.30/s avg_rate=31.94/s
2026-11-09T07:08:18.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:08:18 workers=100 processed=3400 in_flight=31 rate_10s=35.70/s avg_rate=32.30/s
2026-11-09T07:08:28.901000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:08:28 workers=100 processed=3431 in_flight=0 rate_10s=3.10/s avg_rate=29.76/s
2026-11-09T07:08:38.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:08:38 workers=100 processed=3431 in_flight=0 rate_10s=0.00/s avg_rate=27.39/s
```

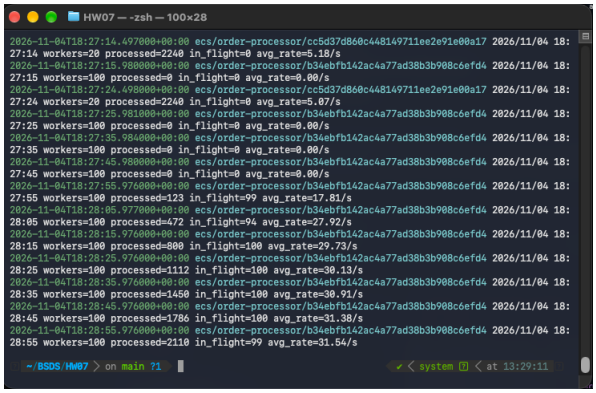
Santosh · Fig 12a. 100 workers: in_flight 94–100, rate_10s 30–35.7/s, ending at processed=3431, in_flight=0



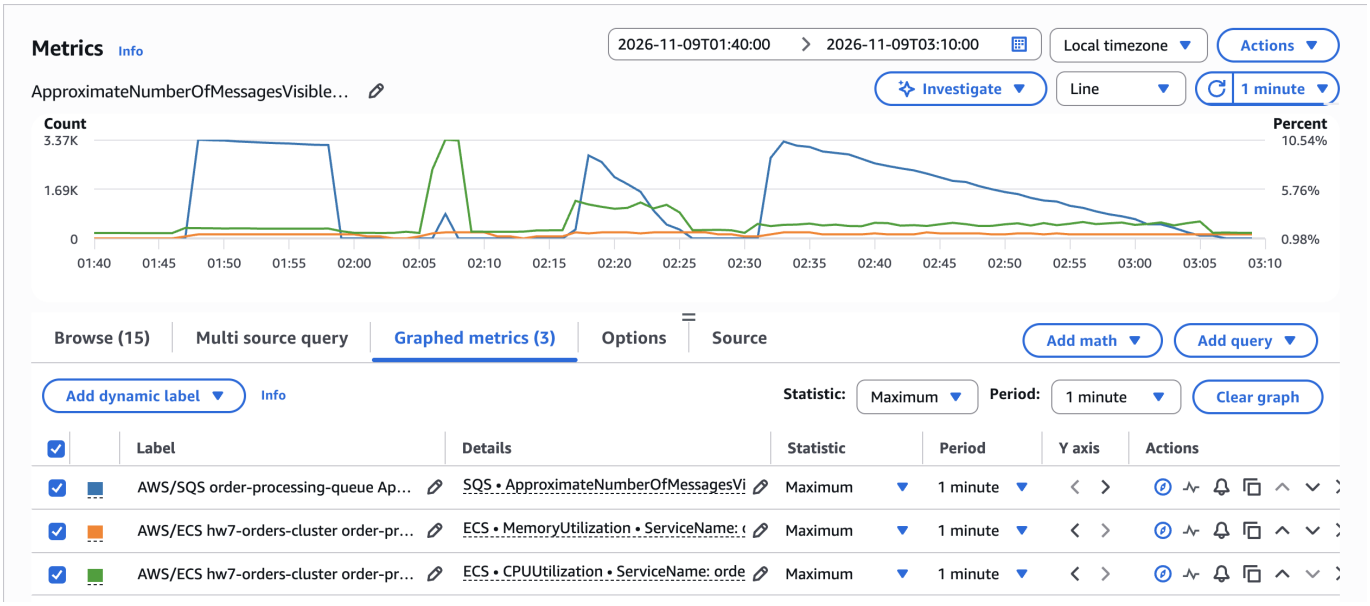
Sameep · Fig 10b. 5 workers: avg_rate 1.62–1.65/s, in_flight=5



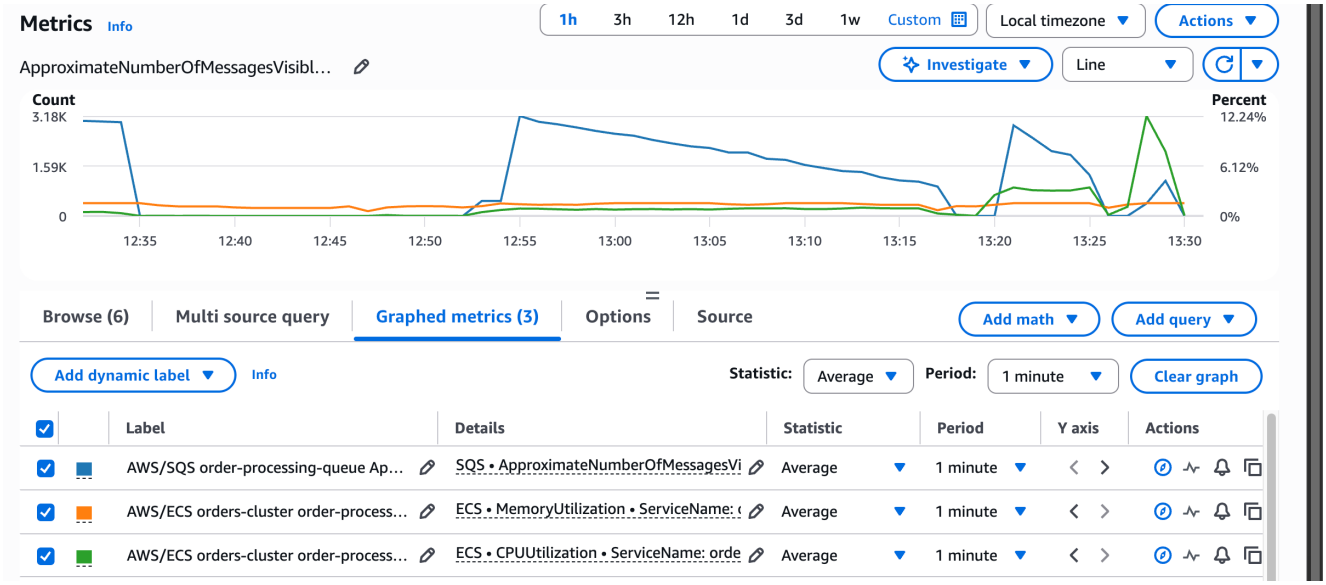
Sameep · Fig 11b. 20 workers: avg_rate ~6.4–6.5/s



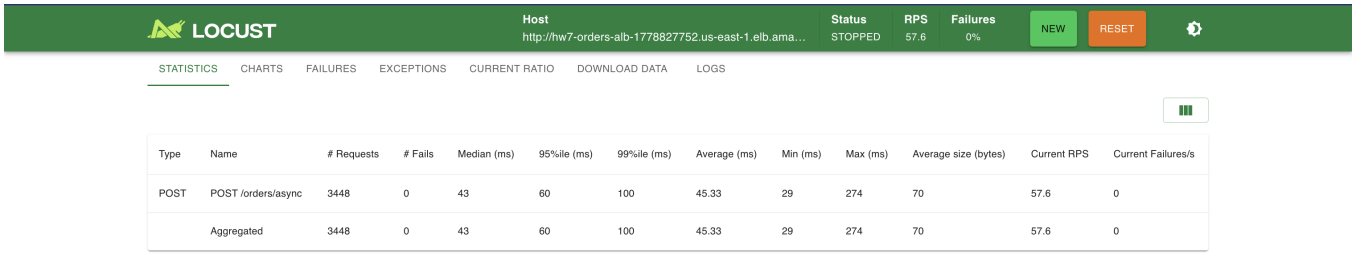
Sameep · Fig 12b. 100 workers: in_flight 99–100, avg_rate up to 31.54/s



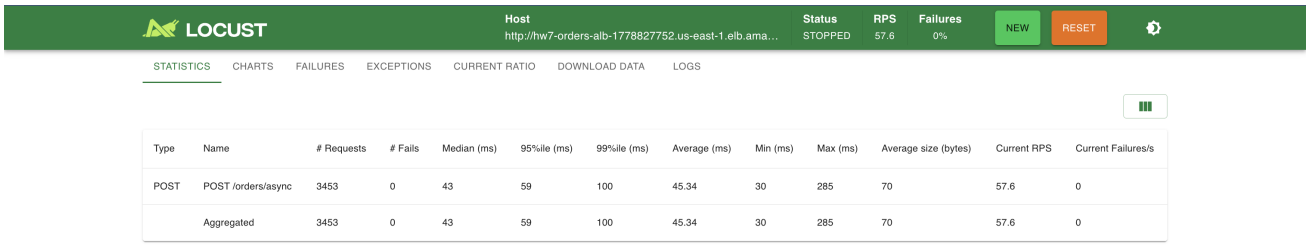
Santosh · Fig 13a. Queue depth, CPU and memory across all four runs (01:40–03:10 local): 1 worker (01:48), 100 workers (02:07), 20 workers (02:17), 5 workers (02:33)



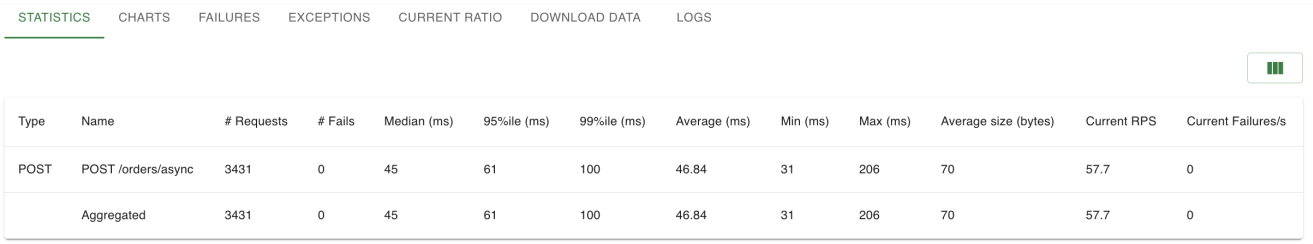
Sameep · Fig 13b. Queue depth, memory and CPU across the 5, 20 and 100 worker runs; CPU peaks at 12.24% during the 100-worker run



Santosh · Locust stats, 5 workers: 3448 requests, 0 failures, median 43 ms, p95 60 ms



Santosh · Locust stats, 20 workers: 3453 requests, 0 failures, median 43 ms, p95 59 ms



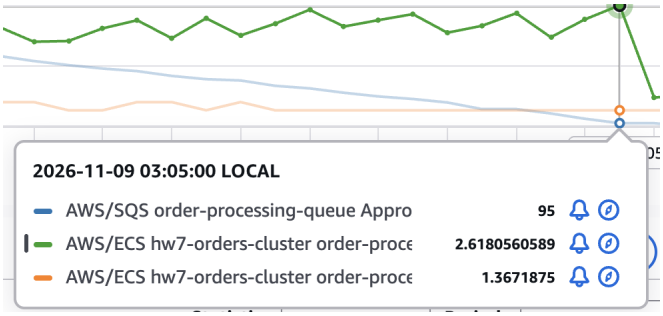
Santosh · Locust stats, 100 workers: 3431 requests, 0 failures, median 45 ms, p95 61 ms

Resource utilization

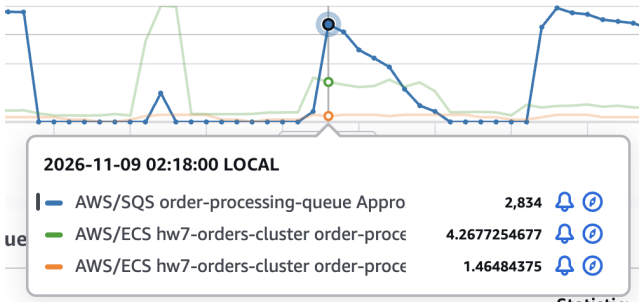
Workers	Santosh: peak CPU / memory (CloudWatch Maximum, 1 min)	Sameep: CPU / memory
1	1.9163025618% / 1.3671875% (01:53)	~0.5% / ~1.3–1.5%
5	2.6180560589% / 1.3671875% (03:05)	~0.8% / ~1.3–1.5%
20	4.2677254677% / 1.46484375% (02:18)	~3.5% peak / flat
100	10.541211128% / 1.5625% (02:07)	~12.2% peak / flat



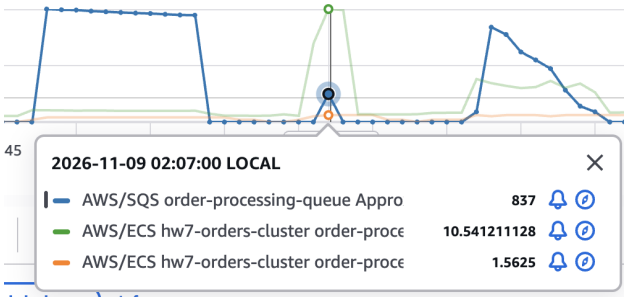
Santosh · 1 worker at 01:53: CPU 1.92%, memory 1.37%



Santosh · 5 workers at 03:05: CPU 2.62%, memory 1.37%



Santosh · 20 workers at 02:18: CPU 4.27%, memory 1.46%



Santosh · 100 workers at 02:07: CPU 10.54%, memory 1.56%

- CPU rises with worker count but stays small for both of us: even 100 goroutines use only ~10–12% of a quarter vCPU, because workers spend nearly all their time in time.Sleep, like a real payment API call waiting on the network.

- Memory stays at ~1.3–1.6%. A goroutine costs only a few KB.
- The bottleneck is concurrency, not CPU or memory. More goroutines fit in the same small task at little cost.

Minimum workers

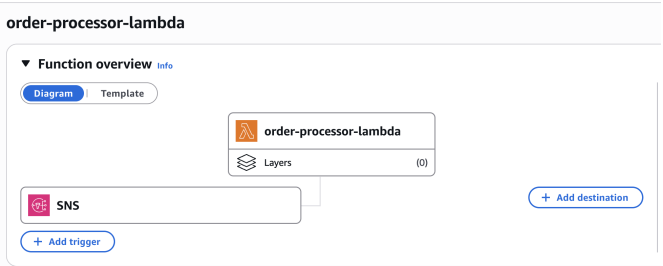
To prevent buildup at 60 orders/s with 3 s per order: **60 × 3 = 180 workers** (theoretical). Both data sets agree that 100 is not enough: Santosh measured 33.22/s against 3431 ÷ 60 = 57.18/s arrivals (queue still grew to 1352); Sameep measured ~31.5/s against ~56/s (queue grew to ~1,250). In both cases the queue only drained after load stopped.

Comparison: measured rates match theory closely for both of us. Santosh's 100-worker rate is closer to theory (33.22 vs ~31.5/s), likely because his poll loop only asks SQS for as many messages as there are free workers, which kept in_flight at 94–100. Peak depths differ by method: Santosh read a 10 s CLI loop, while CloudWatch's 1-minute points miss short spikes (Santosh's 100-worker spike showed 837 in CloudWatch vs 1352 in the CLI). Santosh measured full drains for 5 and 20 workers, filling the two values Sameep left projected or not measured. Santosh's 1- and 5-worker CPU is higher than Sameep's, partly because Santosh graphed the Maximum statistic and Sameep the Average, and likely also from the poll loop and 10 s logging overhead; at 100 workers the payment goroutines dominate and the results converge.

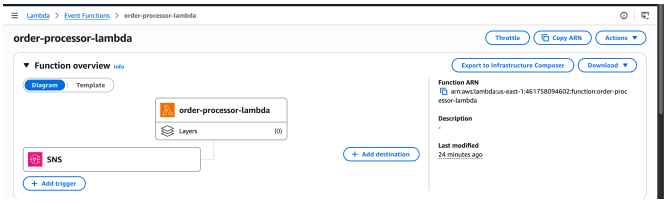
7. Part III: Lambda

Setting	Santosh	Sameep
Function	order-processor-lambda (Go)	order-processor-lambda (Go)
Runtime	provided.al2, x86_64, handler bootstrap	provided.al2, x86_64
Memory / timeout	512 MB / 10 s	512 MB / 10 s
Role	LabRole	LabRole
Trigger	SNS order-processing-events (direct, no SQS)	SNS order-processing-events (direct, no SQS)
Safety control	count = var.enable_lambda ? 1 : 0 on the subscription and SNS permission	count = var.enable_lambda_subscription ? 1 : 0 (default false)
Orders sent	9 by curl	7 by curl

Both of us enabled the subscription only for the curl tests and removed it right after. No load testing was run against Lambda. During the curl tests each order also reached SQS and the ECS processor, because SNS delivers to every subscriber (fan-out).



Santosh · Fig 14a. Lambda console with SNS as the trigger



Sameep · Fig 14b. Lambda console with SNS as the trigger

Cold start observations: Santosh (9 orders)

#	Batch	Log time (UTC)	Instance	Duration	Billed	Init	Type
1	A: first call	08:21:42	b65d41aa	3004.41 ms	3080 ms	74.68 ms	Cold
2	B: one at a time, ~5 s apart	08:21:57	b65d41aa	3001.70 ms	3002 ms	—	Warm
3	B	08:22:03	b65d41aa	3001.70 ms	3002 ms	—	Warm

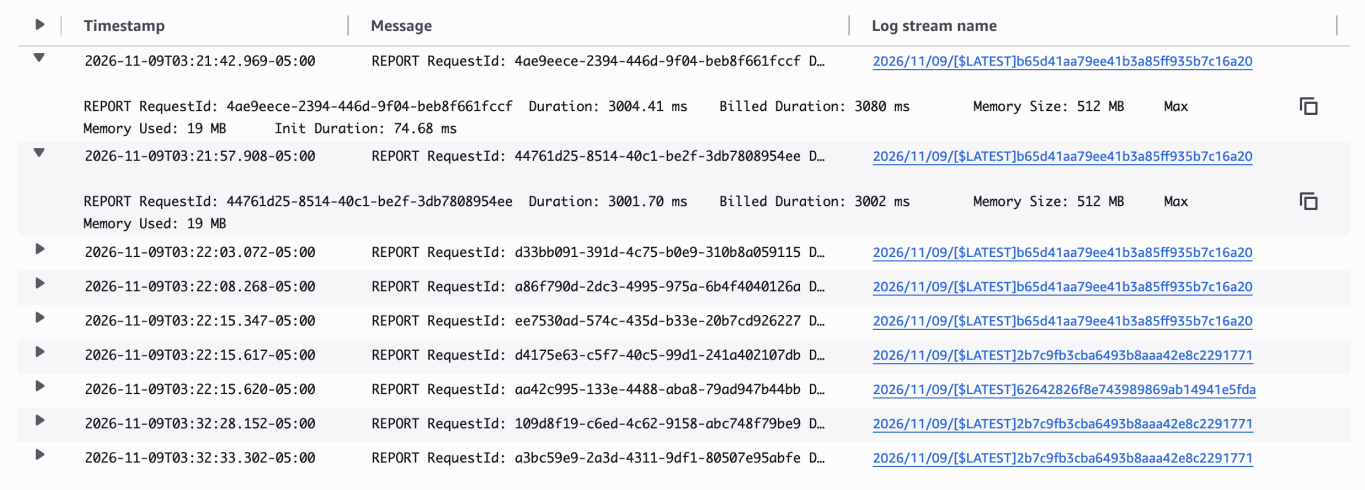
#	Batch	Log time (UTC)	Instance	Duration	Billed	Init	Type
4	B	08:22:08	b65d41aa	3002.23 ms	3003 ms	—	Warm
5	C: 3 at once	08:22:15	b65d41aa	3002.11 ms	3003 ms	—	Warm
6	C	08:22:15	2b7c9fb3	3004.32 ms	3075 ms	70.15 ms	Cold
7	C	08:22:15	62642826	3002.49 ms	3074 ms	70.64 ms	Cold
8	D: after ~10 min idle	08:32:28	2b7c9fb3	3002.28 ms	3003 ms	—	Warm
9	D	08:32:33	2b7c9fb3	3001.88 ms	3002 ms	—	Warm

Instance = log stream ID. Max Memory Used was 19 MB for all 9 invocations.

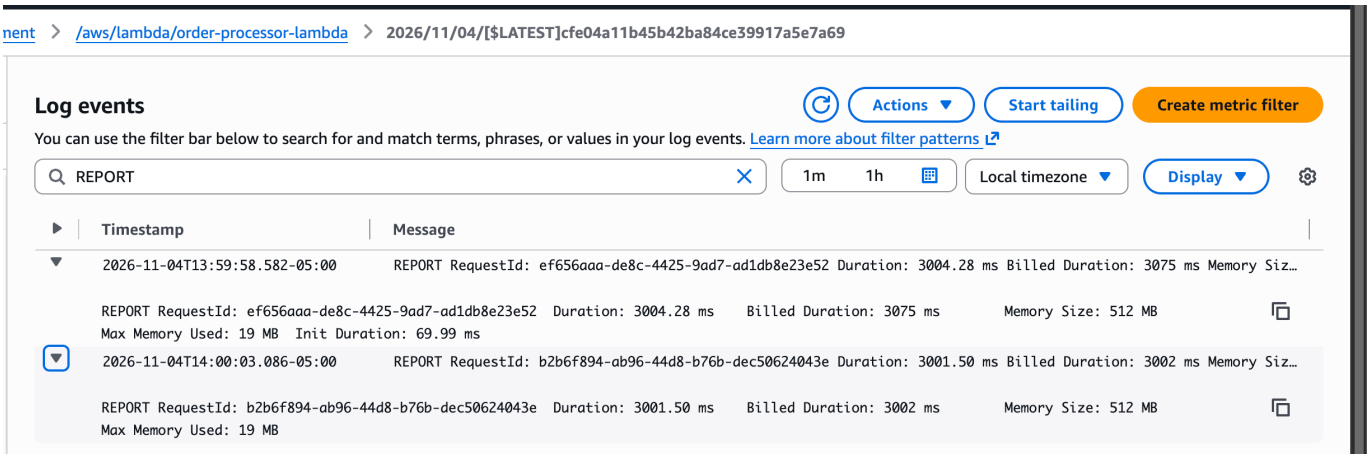
Cold start observations: Sameep (7 orders)

#	When	Duration	Billed	Init	Type
1	First order	3002.77 ms	3075 ms	71.58 ms	Cold
2	Sent 2 s later (instance 1 busy)	3004.64 ms	3094 ms	88.72 ms	Cold
3		3001.74 ms	3002 ms	—	Warm
4		3001.58 ms	3002 ms	—	Warm
5		3001.54 ms	3002 ms	—	Warm
6	After ~10 min idle	3004.28 ms	3075 ms	69.99 ms	Cold
7	5 s after order 6	3001.50 ms	3002 ms	—	Warm

Max Memory Used: 19 MB.



Santosh · Fig 15a. REPORT lines: order 1 cold (Init 74.68 ms, billed 3080 ms) and order 2 warm (billed 3002 ms); log streams show 3 instances



Sameep · Fig 15b. REPORT lines for orders 6 (cold, Init 69.99 ms, billed 3075 ms) and 7 (warm, billed 3002 ms)

Findings

Finding	Santosh	Sameep
Cold start frequency	3 of 9	3 of 7
Cause: first request	Yes (order 1)	Yes (order 1)
Cause: concurrency	Yes: 3 at once → 2 new instances (1 → 3 instances, no configuration)	Yes: order 2 arrived while instance 1 was busy
Cause: idle timeout	No: still warm after ~10 min 13 s idle (instance 2b7c9fb3)	Yes: cold after ~10 min idle
Init overhead	70.15–74.68 ms on ~3000 ms = 2.34–2.49%	69.99–88.72 ms = ~2.3–3.0%
Billing	Init is billed: 3074–3080 ms cold vs 3002–3003 ms warm	Init is billed: 3075–3094 ms cold vs 3002 ms warm
Memory	19 MB of 512 MB (~27× over-provisioned)	19 MB of 512 MB (~27× over-provisioned)

- **Customer impact (both):** none. The customer already got HTTP 202 in ~45 ms; a cold start only delays background processing by under 0.1 s. Both overheads match the assignment's 73 ms / 2.4% example.
- **Memory (both):** 128 MB would cut GB-second cost by 75%, since the job only sleeps.

Comparison: the causes line up except idle. AWS does not guarantee how long an idle instance stays warm, so one of us got a cold start after ~10 minutes and the other did not. Both results are valid; "cold after ~5+ minutes idle" is a rough guide, not a rule.

8. Cost analysis

All values in this section are calculated, not measured. Pricing: $0.20 per million requests, $0.0000166667 per GB-second. Free tier: 1M requests and 400,000 GB-seconds per month. Per order at 512 MB: 3 s × 0.5 GB = 1.5 GB-seconds.

Scenario	Calculation	Result
Santosh: 50,000 orders/month	75,000 GB-s and 50,000 requests, both in free tier	$0 (vs $8.50–$17 ECS)
Santosh: 50,000 orders/month without free tier	75,000 × 0.0000166667 + 50,000 × 0.0000002	$1.26
Sameep: 10,000 orders/month	15,000 GB-s and 10,000 requests, both in free tier	$0 (vs $17 ECS)
Orders covered by free tier	400,000 ÷ 1.5	≈ 266,667 orders/month
Cost per order beyond free tier	1.5 × 0.0000166667 (+ 0.0000002 above 1M requests)	≈ $0.000025

Scenario	Calculation	Result
One 1-hour flash sale at 60/s	216,000 orders × 1.5 = 324,000 GB-s	Within free tier

Break-even (512 MB, requests under 1M)	Calculation	Orders/month
vs 1 ECS task ($8.50): Lambda replaces only the processor	(8.50 ÷ 0.0000166667 + 400,000) ÷ 1.5	≈ 606,667
vs 2 ECS tasks ($17), as in the handout	(17 ÷ 0.0000166667 + 400,000) ÷ 1.5	≈ 946,667
vs $17 at 128 MB (Sameep)	0.375 GB-s per order; request charges above 1M	≈ 3.7M

- We both get ~947K orders/month for break-even against $17, not the ~1.7M stated in the assignment. The working is shown in case the assignment used other assumptions.
- The receiver task, ALB and NAT gateway are needed in both designs. Lambda replaces only the processor, so $8.50 (one task) is the fairer comparison, which brings break-even down to ~607K.
- Both volumes (10K and 50K) are free. Santosh's 50K has 5.3× headroom to the free-tier ceiling and ~12× to the one-task break-even.

9. Trade-offs and recommendation

Gain with Lambda	Loss with Lambda
No queue or worker tuning. In Part II we both changed workers by hand (1, 5, 20, 100), and even 100 could not keep up.	No queue buffer. In Part II, 3,000+ orders waited safely in SQS for over 30 minutes. SNS → Lambda retries a limited number of times, then drops the order.
Pay per use: $0 at both our volumes vs $8.50–$17 for ECS.	No control over batching or visibility timeout.
Automatic scaling: concurrent orders got new instances with no configuration.	Cold starts: ~70–89 ms (2.3–3.0%), and init time is billed.
	Account concurrency limits can throttle very large bursts.

- **Switch the processor to Lambda** at our expected volumes (10K–50K orders/month): $0 vs $8.50–$17, no worker scaling, no queue-depth alerts.
- **Cold starts are not a concern:** processing is already async and takes 3 s, and the customer already has a 202.
- **Keep SQS in front of Lambda (SNS → SQS → Lambda)** so failed orders are not lost. This keeps the 4-day buffer and retries with no workers to manage.
- **Re-evaluate around 600K orders/month** (where Lambda passes the cost of the one ECS task it replaces; ~947K against two tasks). Dropping to 128 MB pushes this further out, since the function uses only 19 MB.